

go-zkfp Development Guide

Go Binding for ZKFinger Reader SDK (libzkfp.dll)

github.com/elmyrockers/go-zkfp

Based on ZKTeco ZKFinger Reader SDK C API Version 2.0

Contents

1	Overview	3
2	Disclaimer	3
3	System Requirements	3
4	Installation and Deployment	3
5	Description of Go Interfaces	3
5.1	Design Conventions	3
5.2	Function Mapping	4
5.3	Type Definitions and Constants	4
6	Global Library Functions	5
6.1	Init	5
6.2	Terminate	5
6.3	GetDeviceCount	5
6.4	OpenDevice	5
6.5	DBInit	5
7	Device Methods	6
7.1	Device.Close	6
7.2	Device.ImageSize	6
7.3	Device.AcquireFingerprint	6
7.4	Device.AcquireFingerprintImage	6
7.5	Device.GetParameter	7
7.6	Device.SetParameter	7
8	Database/Algorithm Methods	7
8.1	DB.Free	7
8.2	DB.Merge	7
8.3	DB.Add	8
8.4	DB.Del	8
8.5	DB.Clear	8
8.6	DB.Count	8
8.7	DB.Identify	9
8.8	DB.Match	9
8.9	DB.ExtractFromImage	9
9	Usage Example	9

10	Appendixes	12
10.1	Appendix 1: List of Common Parameter Codes	12
10.2	Appendix 2: Descriptions of Returned Error Values	13
10.3	Appendix 3: Error Handling	13

1 Overview

`go-zkfp` (github.com/elmyrockers/go-zkfp) is a Go binding for `libzkfp.dll`, the ZKFinger Reader SDK from ZKTeco. It allows Go programs to enumerate ZKTeco fingerprint readers, capture images and templates, and register, identify, and match fingerprints. This document describes the Go API, mapping one-to-one to the C functions of the ZKFinger Reader SDK (C API Version 2.0).

2 Disclaimer

`go-zkfp` is an independent open-source wrapper and does not include the ZKTeco SDK binaries. `libzkfp.dll` must be obtained by installing the official ZKFinger SDK from ZKTeco. Use of the SDK is subject to ZKTeco's licensing terms.

3 System Requirements

1. Operating system: Windows XP or a later version
2. Go 1.27.1 or later
3. ZKFinger SDK 5.x / ZKOnline SDK 5.x installed (providing `libzkfp.dll` and reader drivers)
4. The architecture of the Go program (`GOARCH=386` or `amd64`) must match the architecture of `libzkfp.dll`
5. `cgo` and a C compiler are **not** required: the library is loaded at runtime via `golang.org/x/sys/windows` (`windows.NewLazyDLL`) utilizing standard calling conventions required by the SDK.

4 Installation and Deployment

1. Install the ZKFinger driver (ZKFinger SDK 5.x / ZKOnline SDK 5.x), which you can download from <https://www.zkteco.com>.
2. Add the package to your module:

```
go get github.com/elmyrockers/go-zkfp
```

3. Make `libzkfp.dll` loadable by placing it next to your executable or in a directory on your system PATH. If it cannot be loaded, `Init` fails with `ErrLoadLibrary`.
4. Import the package in your code:

```
import "github.com/elmyrockers/go-zkfp"
```

5 Description of Go Interfaces

5.1 Design Conventions

- **Errors.** Functions return a standard Go **error** instead of raw integer return codes. Non-zero SDK results return values of type `zkfp.Error` (see Appendix 2), allowing direct comparison with `errors.Is` or `errors.As`.

- **Handles.** C HANDLE types are encapsulated into clean Go types: `*Device` (for reader instances) and `*DB` (for algorithm caches/databases).
- **Buffers.** C pointer-and-size parameter pairs are represented as native Go slices (`[]byte`). Output buffers are automatically allocated and trimmed to the actual data size returned by the SDK.
- **Cleanup.** `Close()` and `Free()` methods are implemented safely and idempotently.
- **Closed or nil handles.** All methods return `ErrInvalidHandle` on a closed or nil `Device/DB`, and `ErrInvalidParam` for empty buffers or paths.
- **Template size.** Templates are at most 2048 bytes (`maxTemplateSize`).
- **Order of use.** `Init` → `OpenDevice` → `DBInit` → ... → `DB.Free` → `Device.Close` → `Terminate`.

5.2 Function Mapping

C function	Go function / method
ZKFPM_Init	zkfp.Init
ZKFPM_Terminate	zkfp.Terminate
ZKFPM_GetDeviceCount	zkfp.GetDeviceCount
ZKFPM_OpenDevice	zkfp.OpenDevice
ZKFPM_CloseDevice	(<code>*Device</code>).Close
ZKFPM_SetParameters	(<code>*Device</code>).SetParameter
ZKFPM_GetParameters	(<code>*Device</code>).GetParameter
ZKFPM_AcquireFingerprint	(<code>*Device</code>).AcquireFingerprint
ZKFPM_AcquireFingerprintImage	(<code>*Device</code>).AcquireFingerprintImage
ZKFPM_DBInit	zkfp.DBInit
ZKFPM_DBFree	(<code>*DB</code>).Free
ZKFPM_DBMerge	(<code>*DB</code>).Merge
ZKFPM_DBAdd	(<code>*DB</code>).Add
ZKFPM_DBDel	(<code>*DB</code>).Del
ZKFPM_DBClear	(<code>*DB</code>).Clear
ZKFPM_DBCount	(<code>*DB</code>).Count
ZKFPM_DBIdentify	(<code>*DB</code>).Identify
ZKFPM_DBMatch	(<code>*DB</code>).Match
ZKFPM_ExtractFromImage	(<code>*DB</code>).ExtractFromImage

5.3 Type Definitions and Constants

```
package zkfp

type Device struct { /* unexported */ }
type DB struct { /* unexported */ }
type Error int32

const maxTemplateSize = 2048
```

Exported constants are also provided for parameter codes (`ParamImageWidth`, `ParamGreenLED`, and so on; see Appendix 1) and for error values (`ErrNoDevice`, `ErrCaptureFailed`, and so on; see Appendix 2).

6 Global Library Functions

6.1 Init

[Function] `func Init() error`

[Purpose] Initializes the ZKFinger SDK and loads `libzkfp.dll`. Must be called once before invoking other methods.

[Parameter Description] None

[Return Value]

- `nil`: Succeeded (including code 1 indicating already initialized)
- `error`: Failed, e.g. `ErrLoadLibrary` if the DLL cannot be loaded (See Appendix 2)

6.2 Terminate

[Function] `func Terminate() error`

[Purpose] Releases ZKFinger SDK resources. Close all devices and free all DBs first.

[Parameter Description] None

[Return Value]

- `nil`: Succeeded
- `error`: Failed (See Appendix 2)

6.3 GetDeviceCount

[Function] `func GetDeviceCount() (int, error)`

[Purpose] Returns the number of connected fingerprint devices.

[Parameter Description] None

[Return Value]

- `int, nil`: Connected device count (≥ 0)
- `0, error`: Failed to query device count (See Appendix 2)

6.4 OpenDevice

[Function] `func OpenDevice(index int) (*Device, error)`

[Purpose] Opens a fingerprint device by its zero-based index.

[Parameter Description]

- `index`: Zero-based device index integer

[Return Value]

- `*Device, nil`: Opened device handle instance
- `nil, error`: Failed to open device (returns `ErrOpenDevice`)

6.5 DBInit

[Function] `func DBInit() (*DB, error)`

[Purpose] Creates an algorithm cache/database instance.

[Parameter Description] None

[Return Value]

- `*DB, nil`: Initialized algorithm database handle instance
- `nil, error`: Initialization failed

7 Device Methods

7.1 Device.Close

[Method] func (d *Device) Close() error

[Purpose] Closes the fingerprint device handle.

[Parameter Description] None

[Return Value]

- nil: Succeeded
- error: Failed to close device handle (See Appendix 2)

7.2 Device.ImageSize

[Method] func (d *Device) ImageSize() (int, error)

[Purpose] Queries the required fingerprint image buffer size from the device.

[Parameter Description] None

[Return Value]

- int, nil: Required image buffer size in bytes
- 0, error: Failed to retrieve size parameter (See Appendix 2)

7.3 Device.AcquireFingerprint

[Method] func (d *Device) AcquireFingerprint(image []byte) ([]byte, error)

[Purpose] Captures a fingerprint image and extracts its template.

[Parameter Description]

- image: Pre-allocated byte slice buffer to store the scanned fingerprint image (sized using ImageSize)

[Return Value]

- []byte, nil: Extracted fingerprint template slice trimmed to actual size
- nil, error: Capture or extraction failed. `ErrCaptureFailed` is returned while no finger is on the sensor, so retry after a short delay (see Usage Example)

7.4 Device.AcquireFingerprintImage

[Method] func (d *Device) AcquireFingerprintImage(image []byte) error

[Purpose] Captures a raw fingerprint image without template extraction.

[Parameter Description]

- image: Pre-allocated byte slice buffer to receive the raw image data

[Return Value]

- nil: Succeeded
- error: Image capture failed. `ErrCaptureFailed` is returned while no finger is on the sensor (See Appendix 2)

7.5 Device.GetParameter

[Method] `func (d *Device) GetParameter(code int, size int) ([]byte, error)`

[Purpose] Reads a device parameter code value.

[Parameter Description]

- `code`: Integer parameter code (See Appendix 1)
- `size`: Allocation buffer size based on parameter type (e.g., 4 for integer, larger for strings)

[Return Value]

- `[]byte, nil`: Raw parameter bytes trimmed to size reported by SDK
- `nil, error`: Parameter query failed (See Appendix 2)

7.6 Device.SetParameter

[Method] `func (d *Device) SetParameter(code int, value []byte) error`

[Purpose] Writes or configures a device parameter code value.

[Parameter Description]

- `code`: Integer parameter code (See Appendix 1)
- `value`: Raw parameter data bytes (e.g., 4-byte little-endian integer)

[Return Value]

- `nil`: Succeeded
- `error`: Parameter update failed (See Appendix 2)

8 Database/Algorithm Methods

8.1 DB.Free

[Method] `func (db *DB) Free() error`

[Purpose] Releases the algorithm cache resources.

[Parameter Description] None

[Return Value]

- `nil`: Succeeded
- `error`: Failed to release cache (See Appendix 2)

8.2 DB.Merge

[Method] `func (db *DB) Merge(template1 []byte, template2 []byte, template3 []byte) ([]byte, error)`

[Purpose] Combines three fingerprint templates into a single registered template.

[Parameter Description]

- `template1`: First capture template byte slice
- `template2`: Second capture template byte slice
- `template3`: Third capture template byte slice

[Return Value]

- `[]byte, nil`: Merged template slice trimmed to final size
- `nil, error`: Merge operation failed (e.g., `ErrMergeFailed`)

8.3 DB.Add

[Method] func (db *DB) Add(fid uint, template []byte) error

[Purpose] Adds a registered fingerprint template to the algorithm cache with an associated ID.

[Parameter Description]

- fid: Unsigned integer fingerprint ID (> 0)
- template: Registered template byte slice

[Return Value]

- nil: Succeeded
- error: Failed to add template (See Appendix 2)

8.4 DB.Del

[Method] func (db *DB) Del(fid uint) error

[Purpose] Deletes a fingerprint template from the algorithm cache by its ID.

[Parameter Description]

- fid: Unsigned integer fingerprint ID to delete

[Return Value]

- nil: Succeeded
- error: Failed to delete template (See Appendix 2)

8.5 DB.Clear

[Method] func (db *DB) Clear() error

[Purpose] Removes all fingerprint templates from the algorithm cache.

[Parameter Description] None

[Return Value]

- nil: Succeeded
- error: Failed to clear cache (See Appendix 2)

8.6 DB.Count

[Method] func (db *DB) Count() (int, error)

[Purpose] Returns the total number of fingerprint templates stored in the algorithm cache.

[Parameter Description] None

[Return Value]

- int, nil: Template count integer
- 0, error: Failed to fetch count (See Appendix 2)

8.7 DB.Identify

[Method] func (db *DB) Identify(template []byte) (uint, uint, error)

[Purpose] Performs a 1:N fingerprint comparison against all templates in the cache.

[Parameter Description]

- template: Query fingerprint template byte slice

[Return Value]

- fid, score, nil: Matched fingerprint ID and comparison score (both uint)
- 0, 0, error: Identification failed or no match found (See Appendix 2)

8.8 DB.Match

[Method] func (db *DB) Match(template1, template2 []byte) (int, error)

[Purpose] Compares two fingerprint templates directly (1:1 matching).

[Parameter Description]

- template1: First fingerprint template byte slice
- template2: Second fingerprint template byte slice

[Return Value]

- int, nil: Comparison match score integer (≥ 0)
- 0, error: Comparison calculation failed (See Appendix 2)

8.9 DB.ExtractFromImage

[Method] func (db *DB) ExtractFromImage(path string, dpi uint) ([]byte, error)

[Purpose] Extracts a fingerprint template from an image file (BMP or JPG format).

[Parameter Description]

- path: Full file system path string to the image file. The path must be ASCII, because the SDK expects an ANSI string
- dpi: Image resolution DPI unsigned integer

[Return Value]

- []byte, nil: Extracted template byte slice trimmed to actual size
- nil, error: Extraction failed or unsupported version (See Appendix 2)

9 Usage Example

```
package main

import (
    "errors"
    "fmt"
    "log"
    "time"

    "github.com/elmyrockers/go-zkfp"
```

```

)

func main() {
    // 1. Initialize the SDK
    if err := zkfp.Init(); err != nil {
        log.Fatalf("Init failed: %v", err)
    }
    defer zkfp.Terminate()

    // 2. Initialize the algorithm database/cache for matching & storage
    db, err := zkfp.DBInit()
    if err != nil {
        log.Fatalf("DBInit failed: %v", err)
    }
    defer db.Free()

    // 3. Check for connected devices
    count, err := zkfp.GetDeviceCount()
    if err != nil || count == 0 {
        log.Fatalf("No device found: %v", err)
    }

    // 4. Open the first device
    dev, err := zkfp.OpenDevice(0)
    if err != nil {
        log.Fatalf("OpenDevice failed: %v", err)
    }
    defer dev.Close()

    fmt.Println("Device opened and Algorithm DB initialized successfully
    !")

    // 5. Query required image buffer size
    imgSizeBytes, err := dev.ImageSize()
    if err != nil {
        log.Fatalf("Failed to get image size: %v", err)
    }
    imgBuf := make([]byte, imgSizeBytes)

    fmt.Println("Please place your finger on the scanner...")

    // 6. Poll until a finger is captured
    var template []byte
    for {
        template, err = dev.AcquireFingerprint(imgBuf)

        // No finger on the sensor yet: wait a moment and try again
        if errors.Is(err, zkfp.ErrCaptureFailed) {
            time.Sleep(100 * time.Millisecond)
            continue
        }
        // Any other error is a real failure
        if err != nil {
            log.Fatalf("Fingerprint capture failed: %v", err)
        }
        break
    }
}

```

```
// 7. Add the captured template to the algorithm database with ID 1
var fingerprintID uint = 1
if err := db.Add(fingerprintID, template); err != nil {
    log.Fatalf("Failed to add template to DB: %v", err)
}

totalCount, _ := db.Count()
fmt.Printf("Fingerprint successfully registered! Total records in DB
: %d\n", totalCount)
}
```

10 Appendixes

10.1 Appendix 1: List of Common Parameter Codes

`(*Device).GetParameter` and `(*Device).SetParameter` take an SDK parameter code. Integer parameters are 4-byte little-endian values.

Constant	Code	Access	Type	Description
<code>ParamImageWidth</code>	1	Read	Int	Fingerprint image width in pixels.
<code>ParamImageHeight</code>	2	Read	Int	Fingerprint image height in pixels.
<code>ParamImageDPI</code>	3	Read/Write	Int	Image DPI (LIVEID20R only; 750/1000 recommended for children).
<code>ParamWhiteLED</code>	101	Write	Int	White LED on the reader (1 = blink, 0 = off).
<code>ParamGreenLED</code>	102	Write	Int	Green LED on the reader (1 = blink, 0 = off).
<code>ParamRedLED</code>	103	Write	Int	Red LED on the reader (1 = blink, 0 = off).
<code>ParamBuzzer</code>	104	Write	Int	Buzzer (1 = start buzzing, 0 = off).
<code>ParamImageSize</code>	106	Read	Int	Fingerprint image buffer size in bytes.
<code>ParamVIDPID</code>	1015	Read	4-byte array	USB VID and PID (first two bytes are the VID, last two bytes the PID).
<code>ParamVendorInfo</code>	1101	Read	String	Vendor information.
<code>ParamProductName</code>	1102	Read	String	Product name.
<code>ParamSerialNumber</code>	1103	Read	String	Device serial number (SN).
<code>ParamAntiFake</code>	2002	Read/Write	Int	Anti-fake function (LIVEID20R only; 1 = enable, 0 = disable).
<code>ParamFakeStatus</code>	2004	Read	Int	True if the lower five bits are all 1's (<code>value&31 == 31</code>).
<code>ParamTemplateFormat</code>	10001	Write	Int	Template format (ISO/ANSI readers only; 0 = ANSI378, 1 = ISO 19794-2).

Reading a parameter

```
value, err := device.GetParameter(zkfp.ParamImageWidth, 4)
if err != nil {
    return err
}

width := int(binary.LittleEndian.Uint32(value))
```

Writing a parameter

```
// Turn the green LED on.
if err := device.SetParameter(zkfp.ParamGreenLED, []byte{1, 0, 0, 0});
    err != nil {
    return err
}
```

`ImageSize()` already wraps `ParamImageSize`, so you rarely need to read it yourself.

Code	Go Error Value	Description
1	<code>ErrAlreadyInit</code>	SDK already initialized (treated as success by <code>Init</code>)
0	<code>nil</code>	Operation succeeded
-1	<code>ErrInitLib</code>	Failed to initialize the algorithm library
-2	<code>ErrInitCapture</code>	Failed to initialize the capture library
-3	<code>ErrNoDevice</code>	No device connected
-4	<code>ErrNotSupported</code>	Not supported by the interface
-5	<code>ErrInvalidParam</code>	Invalid parameter
-6	<code>ErrOpenDevice</code>	Failed to start the device
-7	<code>ErrInvalidHandle</code>	Invalid handle
-8	<code>ErrCaptureFailed</code>	Failed to capture the image
-9	<code>ErrExtractFailed</code>	Failed to extract the fingerprint template
-10	<code>ErrAbort</code>	Suspension operation
-11	<code>ErrNoMemory</code>	Insufficient memory
-12	<code>ErrBusy</code>	Device is busy capturing
-13	<code>ErrAddFailed</code>	Failed to add template to memory
-14	<code>ErrDeleteFailed</code>	Failed to delete template
-17	<code>ErrOperationFailed</code>	Operation failed (other error)
-18	<code>ErrCaptureCancelled</code>	Capture cancelled
-20	<code>ErrMatchFailed</code>	Fingerprint comparison failed
-22	<code>ErrMergeFailed</code>	Failed to combine registered templates
-23	<code>ErrOpenFile</code>	Opening the file failed
-24	<code>ErrImageProcess</code>	Image processing failed
-100	<code>ErrLoadLibrary</code>	Failed to load <code>libzkfp.dll</code> (ZKFinger driver)

10.2 Appendix 2: Descriptions of Returned Error Values

10.3 Appendix 3: Error Handling

Every function that talks to the SDK returns a `zkfp.Error`, an `int32` that holds the original `libzkfp.dll` result code.

```
type Error int32
```

Basic check. Most calls only need a normal `err != nil` check:

```
if err := zkfp.Init(); err != nil {
    log.Fatal(err)
}
```

Specific errors. Compare against a constant with `errors.Is`. This is the usual way to wait for a finger, because `ErrCaptureFailed` only means nothing is on the sensor yet:

```
for {
    err := device.AcquireFingerprintImage(buf)
    if err == nil {
        break
    }
    if !errors.Is(err, zkfp.ErrCaptureFailed) {
        return err
    }
    time.Sleep(100 * time.Millisecond)
}
```

Raw code. Use `errors.As` to read the SDK result code:

```
var zkErr zkfp.Error

if errors.As(err, &zkErr) {
    fmt.Println("ZKFinger error code:", int32(zkErr))
}
```

Messages. Each message contains a description and the SDK code:

```
zkfp: failed to initialize the algorithm library (-1)
```

A non-zero result code that has no constant is still returned as a `zkfp.Error`:

```
zkfp: unknown error (<code>)
```